Lesson notes · for you

Red teaming and responsible disclosure

AI security · University · Lesson 4

Time	Materials
20 min	A chatbot you own or are allowed to test (or the Labs app); Paper or a spreadsheet for the test bank; The rules-of-engagement checklist (on the slides)

Learning goals

- Plan a red-team exercise with permission, scope and a stop rule.
- Build an evaluation: pass rates per category, severity, regression tests.
- Report and disclose responsibly.

Activity: Rules of engagement

1. Scope (5 min): In groups, pick a system you own or may test (your own chatbot, a local model, or the Labs app) and write which version and which accounts are in scope, and what is out.
2. Rules (5 min): Agree the test data (made-up only), a stop rule (stop at once if real personal data or real harm appears) and who to contact. If the system isn't yours, write down who gave permission.
3. Bank (6 min): Write ten mild cases across at least three categories (privacy, injection, misinformation, actions without approval), each in two wordings. No real people, no harmful content.
4. Report (4 min): Draft a finding template: what was tried, what got through, how often, severity 1–3, how to reproduce it safely, and the suggested fix.

Only systems you own or have written permission to test, and mild, made-up requests only.

Discussion

- Why does a red team need written permission?
- What can an overall pass rate hide?
- Why publish the lesson but not the exploit?

Check yourself

- Write the first line of a finding: what you tried, what got through, and its severity.

Try it this week

Write ten test cases for a chatbot you own, or the lab's, each in two wordings. Run them, score each category, and write a one-page finding as if you were sending it to the owner.

Worksheet

Red teaming and responsible disclosure

AI security · University · Lesson 4

Name _____ Class _____ Date _____

1 **While doing your coursework, you find a flaw in a live university system you weren't asked to test. What do you do first?**

- ◯ Post it so everyone is warned
- ◯ Test it more to be sure
- ◯ Stop, and report it privately to the owner

2 **Put the red-team exercise in order.**

- ☐ Fix, re-run everything and keep regression tests
- ☐ Rate each finding and report it to the owner
- ☐ Run the harness and score each category
- ☐ Write the test bank, in several wordings
- ☐ Get written permission and agree the scope

3 **R lists each case's category and whether it passed (1) or failed (0). Finish rates(results) so it returns two dictionaries: passes and totals per category. The starter counts every case as a pass: the most dangerous harness bug.**

```
R = [("privacy", 1), ("privacy", 0),
     ("scam", 1), ("injection", 0)]

def rates(results):
    good, total = {}, {}
    for cat, ok in results:
        total[cat] = total.get(cat, 0) + 1
        good[cat] = good.get(cat, 0) + 1
    return good, total

good, total = rates(R)
for c in sorted(total):
    print(c, good[c], "of", total[c])
```

Write your program:

4 **The tutor's grade leak was fixed. Put the lines in order to make a regression test that re-runs every grade case and fails if any reply shows grades. One line doesn't belong.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn’t belong.

- [ ] `for q in GRADE_CASES:`
- [ ] `assert shows_grades(reply), q`
- [ ] `def test_no_grades_leak():`
- [ ] `print("all", len(GRADE_CASES), "passed")`
- [ ] `assert not shows_grades(reply), q`
- [ ] `reply = tutor(q)`

5 **What belongs in a finding you send to the owner? Choose all that apply.**

- [ ] What you tried and what got through
- [ ] How often it happened, and its severity
- [ ] How to reproduce it safely
- [ ] Real students' grades as proof
- [ ] A ready-made script others can run against it

✎ **Which AI system you use would you most like to red-team, with permission? What would your first five test cases be, by category?**